

SHUNYA Engineering Constitution — Governance of Evolution

Constitutional Layer: Design Constitution → Engineering Constitution **Status:** Frozen. Governs how SHUNYA evolves. **Governs:** Hermes and every future engineer. **Conforms to:** CANONICAL_ARCHITECTURE.md (V1.0) + SHUNYA_PATTERN_LANGUAGE.md

Preamble

Engineering behaviour shall become **constitutionally deterministic**.

No code is written by instinct. No feature is built in isolation. Every action SHUNYA engineers take must be traceable to a constitutional article, a canonical pattern, a journey, a runtime, an owner, and a Living Object.

Before ANY implementation begins, the engineer must answer every question in this constitution. If any answer is missing, the work is not ready to begin.

Part I — The Pre-Implementation Oath

Before ANY implementation, the engineer must answer all of these:

#	Question	Where the answer lives
1	Which constitutional article?	CANONICAL_ARCHITECTURE.md
2	Which pattern?	SHUNYA_PATTERN_LANGUAGE.md
3	Which journey?	CANONICAL_ARCHITECTURE.md §18
4	Which runtime?	CANONICAL_ARCHITECTURE.md §1
5	Which owner?	CANONICAL_ARCHITECTURE.md §10-11
6	Which Living Object?	CANONICAL_ARCHITECTURE.md §6
7	Which acceptance gate?	CANONICAL_ARCHITECTURE.md §23
8	Which complexity disappears?	SHUNYA_PATTERN_LANGUAGE.md + Experience Metrics
9	Which file becomes unnecessary?	CANONICAL_ARCHITECTURE.md §21
10	Which existing capability evolves?	CANONICAL_ARCHITECTURE.md §20

If the answer to any question is "none" or "I don't know," the implementation must not begin.

Part II — Mandatory Engineering Checkpoints

Checkpoint A: Before CREATING a file

The engineer must demonstrate:

- ☐ The file serves a constitutional purpose (Understanding / Execution / Trust / Adaptation)
- ☐ The file maps to a canonical pattern (if it's a component)
- ☐ The file belongs to exactly one canonical owner (Ownership Law)
- ☐ The file does not duplicate an existing file (check §21 elimination list)
- ☐ The file's state is declared (State Owned / State Forbidden) per Runtime Contract
- ☐ No existing file could be extended instead (LX-06 Rule)
- ☐ The file reduces overall complexity, not increases it

Creating a file is a constitutional act. It requires justification.

Checkpoint B: Before creating an API

- ☐ Which Living Object does this API serve?
- ☐ Which runtime owns it?
- ☐ Does it follow the Universal Object API pattern (/api/v1/objects/{id})?
- ☐ Does an existing API already serve this need?
- ☐ Does it emit events with the Universal Event Law metadata?
- ☐ Is it simple (Constitutional Simplicity Rule)?
- ☐ Does it preserve the Object Interaction Law (causality, evidence, timeline)?

Checkpoint C: Before creating a runtime

- ☐ Does this runtime occupy a position in the constitutional hierarchy (§1)?
- ☐ Is there already a runtime at that position?
- ☐ Can it be an internal capability of an existing runtime instead?
- ☐ Does it have exactly one canonical owner?
- ☐ Is its full contract declared (Purpose, Inputs, Outputs, Events, State, Dependencies, Failure, Recovery)?
- ☐ Does it serve Living Objects, not itself?

No parallel constitutional runtimes may exist. If a position is occupied, extend it.

Checkpoint D: Before creating state

- ☐ Which runtime owns this state (declared in its contract)?
- ☐ Is this state already owned elsewhere?
- ☐ Does it follow the State Fabric pattern (versioned, snapshot-able)?
- ☐ Is it persistent or transient — and is that correct?
- ☐ Is the state declared as State Owned (not State Forbidden)?
- ☐ Does it reduce cognitive load, not add it?

Checkpoint E: Before creating a component

- ☐ Which pattern does it implement? (must be one of the 42 canonical patterns)
- ☐ Which Reality does it visualize?
- ☐ Which Object does it represent?
- ☐ Which AI cognition does it expose?
- ☐ Which execution does it enable?
- ☐ Which next action does it surface?
- ☐ Does it have Failure and Recovery behaviour?
- ☐ Does it avoid the component's Never Allowed list?

Components that merely display data violate the Frontend Constitution.

Checkpoint F: Before creating a button / command

- ☐ Which command does it map to in the Command Surface?
- ☐ Which Living Object does it act on?

- ☐ Which journey does it advance?
- ☐ Is there already a command for this action?
- ☐ Does it preserve context (no hard navigation)?
- ☐ Is its effect visible and reversible (Undo)?

Checkpoint G: Before creating a page

- ☐ **SHUNYA is not page-centric.** Is this truly a page, or a workspace state?
- ☐ If it's a new workspace, does it follow the Workspace Purity Law?
- ☐ Does it preserve navigation continuity?
- ☐ Does it violate the Navigation Constitution?
- ☐ Could it be a state within an existing workspace instead?

Checkpoint H: Before creating polling

- ☐ Is there already a real-time transport (SSE / Delta Events)?
- ☐ Does this duplicate an existing polling loop? (§21 elimination)
- ☐ Does it violate the "no parallel polling" principle?
- ☐ Should it be a subscriber to the canonical event stream instead?
- ☐ Is the polling interval justified?

Checkpoint I: Before creating AI

- ☐ Which runtime owns this AI capability?
- ☐ Does it flow through the canonical AI Provider chain?
- ☐ Does it expose confidence, alternatives, and unknowns (Cognition)?
- ☐ Is it explainable (Trust Explanation pattern)?
- ☐ Does it serve a decision, not replace the founder?
- ☐ Does it duplicate an existing AI capability?

Checkpoint J: Before creating a database table

- ☐ Which Living Object does this table persist?
- ☐ Does it follow the Universal Object Lifecycle?
- ☐ Is there already a table for this object? (check §20-21)
- ☐ Does it have a canonical owner?
- ☐ Does it preserve the Object Interaction Law (audit, rollback)?
- ☐ Is a new table constitutionally justified, or can an existing one extend?

Checkpoint K: Before DELETING code

- ☐ Which canonical owner is delegating this code?
- ☐ Is there a migration plan (Consumers, Providers, Dependencies)?
- ☐ Have all consumers been migrated to the canonical owner?
- ☐ Is there a dependency graph (Runtime, API, Store, Hook, Context deps)?
- ☐ Is there a rollback strategy?
- ☐ Has verification passed (all affected tests + founder walkthrough)?
- ☐ Is this the final step of the convergence (not the first)?

Deletion is the final step, never the first. (LX-06A §4)

Checkpoint L: Before MERGING code

- ☐ Which is the canonical owner of the merged result?
- ☐ Which non-canonical code is being absorbed?
- ☐ Is the merge atomic and independently reversible?
- ☐ Does the merge preserve the constitutional hierarchy?
- ☐ Does the merged result serve Living Objects better than the two parts?
- ☐ Does it reduce complexity (Simplicity Rule)?

Checkpoint M: Before COMMITTING code

- ☐ Does every file pass its relevant checkpoint(s)?
- ☐ Does the change map to a constitutional article?
- ☐ Does the change map to a pattern / journey / object?
- ☐ Does the change reduce complexity?
- ☐ Does the change improve at least one Experience Metric?
- ☐ Does the change introduce no constitutional violation?
- ☐ Is the commit message traceable (article + pattern + journey)?

Part III — The Ten Constitutional Questions

Before any implementation, the engineer writes a short justification covering:

1. **Which constitutional article?** — cite the section
2. **Which pattern?** — cite the pattern number
3. **Which journey?** — cite the canonical journey
4. **Which runtime?** — cite the hierarchy position
5. **Which owner?** — cite the owner
6. **Which Living Object?** — cite the object
7. **Which acceptance gate?** — cite the gate
8. **Which complexity disappears?** — name it
9. **Which file becomes unnecessary?** — name it
10. **Which existing capability evolves?** — name it

This is the Remembrance Check. It ensures every change is a constitutional evolution, not drift.

Part IV — Engineering Invariants

These invariants are permanent and non-negotiable:

#	Invariant	Source
1	Experience is the top constitutional layer	Arch §1
2	Objects are the primary abstraction, not runtimes	Arch §5
3	Every element has exactly one owner	Arch §10
4	No parallel constitutional runtimes	Arch Checkpoint C
5	Workspace is a projection layer only	Arch §16
6	No hard navigation	Arch §15
7	No invisible mutation (Object Interaction Law)	Arch §9

8	No anonymous events (Universal Event Law)	Arch §14
9	No runtime owns undeclared state	Arch §12
10	Simpler architecture prevails	Arch §4
11	Deletion is the final step	Arch LX-06A §4 / Checkpoint K
12	Every component visualizes Reality	Arch §17
13	Every interaction follows the canonical pipeline	Arch §19
14	No duplicate patterns	Pattern Language
15	Every convergence is independently reversible until accepted	Arch §Atomic Execution

Part V — Engineering Verification

After any implementation, the engineer must return to the acceptance gates:

- ☐ Does the change satisfy its acceptance gate(s)? (Arch §23)
- ☐ Does it demonstrate measurable Experience improvement?
- ☐ Did complexity decrease (files, blueprints, runtimes)?
- ☐ Were no regressions introduced (full test suite)?
- ☐ Does the change map cleanly to the canonical architecture?
- ☐ Does the change conform to the pattern language?

If any verification fails, the change is not complete.

Part VI — Forbidden Engineering Behaviours

The following behaviours are constitutionally forbidden:

- **Creating a duplicate** — a file, API, runtime, or component that duplicates a canonical one
- **Building in isolation** — a feature with no constitutional justification
- **Hard nav reliance** — `window.location.href` page jumps in canonical components
- **Silent failure** — pretending a failure was a success
- **Drift** — change that cannot cite an article, pattern, journey, and object
- **Premature deletion** — removing code before migration and verification
- **Uncontrolled merge** — merging without a canonical owner
- **State hoarding** — a runtime owning undeclared state
- **Over-abstraction** — adding a layer the Simplicity Rule forbids
- **Bypassing permissions** — acting without Permission Resolution

Part VII — The Engineering Oath

I will not create before I understand. I will not delete before I migrate. I will not build before I justify. I will not duplicate what is canonical. I will not violate a constitutional invariant, even temporarily. Every change I make shall serve Understanding, Execution, Trust, or Adaptation. Every change I make shall reduce complexity. Every change I make shall make the founder calmer, clearer, and more capable.

Cross-Reference: Engineering ↔ Pattern Language

Engineering Act	Governing Pattern(s)
Create component	Living Object Card, Object Detail, Dashboard
Create action	Quick Action, Command Surface
Create AI	AI Thought, AI Presence, Trust Explanation
Handle error	Error Recovery, Retry
Handle empty	Empty State
Handle loading	Loading
Navigation	Workspace Transition
Notify	Notification
Payments	Payment
Approval	Approval, Approval Queue
Execution	Execution Timeline, Background Execution, Outcome Summary
Conversation	Conversation
Search	Search, Knowledge Card
Onboard	Onboarding

END OF ENGINEERING CONSTITUTION

This document governs engineering behaviour only. It introduces no architecture, no runtime ownership, and no features. It conforms to CANONICAL_ARCHITECTURE.md and SHUNYA_PATTERN_LANGUAGE.md.